

RIFT FIGHTERS

Rapport de soutenance final



Louis SCHWEYCKART

Adam SENOUCI

Ioan DINCOV

Repo :

<https://github.com/Louys007/RiftFighters>

Site internet :

<https://louys007.github.io/RiftFighters/>

Sommaire

Sommaire.....	3
1. Introduction	4
2. Vue générale sur le projet et l'équipe.....	6
2.1 Les membres du groupe	6
2.2 C'est quoi ce projet ?.....	8
2.3 Répartition des tâches.....	12
2.4 L'organisation de ce rapport.....	14
2.5 Outils et logiciels utilisés	14
3. Conception et Architecture du Moteur From Scratch	17
4. Implémentation du Cœur de Jeu : Physique et Réseau P2P	22
4.1 Les mécaniques de combat et la physique.....	22
4.2 L'architecture réseau	28
5. Direction Artistique et Intégration Visuelle	30
5.1. Standardisation des Assets et Identité Visuelle	30
5.2. L'Illusion du Mouvement : Animer avec très peu d'images	31
5.3. Effets Visuels (VFX) et "Game Feel"	32
5.4. L'Ambiance des Arènes (Stages)	34
6. Finitions Techniques : Intelligence Artificielle et Système Sonore	35
6.1 Une Intelligence Artificielle Humanisée (Le Bot)	35
6.2 Le Système Sonore Centralisé	40
7. Bilan du projet.....	42
8. Conclusion	45

1. Introduction

- **Le contexte :**

Ce projet s'inscrit dans le cadre de notre première année du cycle préparatoire à l'EPITA, à travers le module de projet informatique d'envergure SAE J3D. L'objectif principal était de concevoir un jeu vidéo complet afin de mettre en pratique nos connaissances en algorithmique, en programmation orientée objet et en architecture logicielle. Nous avons choisi de développer Rift Fighters, un jeu de combat. Afin de nous confronter aux véritables défis du développement, nous avons pris la décision de coder notre moteur de jeu de A à Z (from scratch) en utilisant la bibliothèque PyGame.

- **Le jeu :**

Rift Fighters s'inscrit dans la pure tradition des jeux de combat en deux dimensions (versus fighting 2D). La particularité technique majeure de ce projet réside dans son implémentation : le jeu n'utilise aucun moteur commercial clé en main (comme Unity, Godot ou Unreal Engine), mais repose sur un moteur propriétaire développé intégralement from scratch en Python 3.11 au-dessus de la bibliothèque multimédia PyGame.

L'objectif principal des joueurs est de s'affronter au cours d'un duel à haute intensité dans une arène fermée, le but étant de réduire à zéro la barre de vie (100 HP) de l'adversaire avant la fin du chronomètre réglementaire. Pour y parvenir, chaque joueur doit maîtriser la gestion de l'espace, le timing des impacts et exploiter le panel unique d'actions de son personnage afin de punir les ouvertures dans la garde adverse.

- **Les obstacles :** Le développement de *Rift Fighters* a été jalonné de plusieurs défis majeurs qui ont mis à l'épreuve notre capacité d'adaptation.

Le premier obstacle technique et humain a été la perte de deux

membres de notre groupe au fil de l'année. Cette réduction importante de notre effectif nous a contraints à redistribuer les tâches en urgence et à revoir certains de nos objectifs initiaux à la baisse afin de garantir la livraison d'un jeu fonctionnel.

Par ailleurs, notre équipe ne comptait aucun graphiste de formation, ce qui représentait un défi considérable pour la création de l'identité visuelle, des sprites et des animations. Pour pallier cette lacune, nous avons dû faire preuve d'ingéniosité en mettant en place un processus créatif inédit : nous avons mis à l'écrit les idées de nos personnages, que nous avons ensuite fournis à l'intelligence artificielle Google Gemini. Cela nous a permis de générer et d'affiner nos ressources visuelles en pixel-art pour obtenir un résultat cohérent.

Enfin, nous avons dû composer avec une gestion du temps extrêmement complexe en raison de la charge de travail académique. La fin d'année a été particulièrement dense, marquée par l'enchaînement de projets exigeants, comme le projet *minimake*, qui se sont cumulés avec la préparation et le passage de tous nos partiels.

- **Le but du rapport :**

Ce document a pour vocation de vous plonger dans les coulisses du développement de *Rift Fighters*. Au fil des pages, nous détaillerons l'évolution technique et créative de notre projet. Nous retracerons notre parcours depuis la création des toutes premières briques de notre moteur physique en Python, jusqu'à l'implémentation complexe de notre architecture réseau en Peer-to-Peer, pour enfin aboutir aux ultimes finitions que sont l'intelligence artificielle et l'habillage sonore.

2. Vue générale sur le projet et l'équipe

2.1 Les membres du groupe

- **Louis SCHWEYCKART** : Responsable de l'architecture réseau et du cœur du moteur, mon principal défi a été de concevoir un système multijoueur fluide sans serveur dédié. J'ai dû plonger dans les spécificités du protocole UDP et coder un algorithme de prédiction client couplé à un système de réconciliation. Ce fut une tâche complexe, parsemée de bugs de désynchronisation, mais parvenir à un gameplay sans latence perçue ("zero input lag") sur un jeu de combat 2D développé de zéro restera ma plus grande fierté technique de cette année. La refonte des menus et l'intégration de l'audio sur la dernière ligne droite ont également permis d'apporter le polissage dont le jeu manquait.
- **Adam SENOUCI** : En charge du moteur physique et de la logique de combat, Adam s'est concentré sur la précision mathématique des affrontements. Il a développé le système de collisions (AABB) et affiné la gestion des *hitboxes* et *hurtboxes* pour que chaque impact soit précis et punitif. Sur cette dernière phase du projet, son travail acharné a permis de donner vie au mode solo grâce à l'implémentation de l'Intelligence Artificielle (BotAI), capable de calculer dynamiquement les distances et d'adapter ses attaques (corps-à-corps ou projectiles) en fonction du comportement du joueur.
- **Ioan DINCOV** : Garant de l'identité visuelle et de la cohérence du *roster*, Ioan a mené à bien l'intégration graphique de nos nouveaux combattants, notamment le Samouraï et le Soldat Médiéval. Son travail ne s'est pas limité à l'animation des *sprites* pour les attaques primaire et secondaire ; il a également ajusté méticuleusement la *frame data* et l'équilibrage des dégâts pour s'assurer que les

affrontements restent justes et compétitifs. En parallèle, il a assuré le développement et la mise à jour de notre site web vitrine.

2.2 C'est quoi ce projet ?

- La genèse :

L'idée de Rift Fighters est née d'une passion au sein de l'équipe, portée plus particulièrement par Adam SENOUCI, un joueur très investi dans la scène des jeux de combat compétitifs (versus fighting). Dès la formation du groupe, notre volonté était claire : nous orienter vers ce genre exigeant. Par pragmatisme, et pour éviter de concevoir un cahier des charges irréalisable qui nous aurait menés à des fausses promesses, nous avons décidé de retourner aux sources. Nous nous sommes longuement renseignés sur les piliers fondateurs du genre, et notamment sur les mécaniques de Street Fighter II. Notre objectif était de créer un jeu dont l'échelle technique restait à notre portée, mais dont la rejouabilité serait virtuellement infinie grâce à la nature même de l'affrontement joueur contre joueur.

- La philosophie de design :

Notre philosophie s'articule autour de l'un des principes les plus nobles du Game Design : « easy to learn, very hard to master » (très facile à prendre en main, très difficile à maîtriser). Rift Fighters a été pensé pour que n'importe quel joueur puisse comprendre les contrôles en quelques secondes, lui conférant un aspect "party game" immédiat. En revanche, pour les joueurs plus investis, la courbe de progression et la profondeur du gameplay se doivent d'être vertigineuses. La différence entre un joueur débutant et un joueur vétérane ne se fait pas sur la complexité des touches, mais sur la gestion des distances (spacing), la lecture du jeu adverse (yomi) et le timing des exécutions.

- Résumé des mécaniques :

La condition de victoire est classique et brutale : il faut vider la barre de vie de son adversaire avant la fin du chronomètre. Pour y parvenir, chaque personnage dispose d'un panel d'actions restreint mais extrêmement polyvalent :

1. Mobilité : Les joueurs peuvent se déplacer latéralement, utiliser un double saut pour les approches aériennes, et un dash (ruée) soumis à un temps de recharge (cooldown) pour esquiver ou réduire rapidement la distance.
2. Offensive : Chaque combattant dispose de deux attaques uniques (généralement un coup rapide au corps-à-corps et un coup lourd ou un projectile), possédant chacune ses propres temps de préparation et de récupération.
3. Défensive et Profondeur : Le joueur peut activer une garde (bouclier) qui réduit les dégâts subis, mais qui l'immobilise pendant un court instant, le rendant vulnérable s'il l'utilise dans le vide. Pour récompenser la prise de risque et ajouter une couche de profondeur tactique au gameplay, nous avons implémenté un système de Perfect Block (parade parfaite) : si le joueur active son bouclier à l'instant exact de l'impact adverse, les dégâts sont annulés et le cooldown de baisse du bouclier est réduit à zéro, lui offrant une fenêtre immédiate pour punir son adversaire.

- Les personnages jouables :

Dans RiftFighters, la diversité du gameplay repose sur quatre archétypes uniques de combattants, chacun représentant une ère différente (Passé, Présent, Futur, Fantastique) et possédant un style de combat qui lui est propre.

Philosophie de Design

La conception de notre roster a été guidée par trois principes fondamentaux :

- ❖ Archétypes clairs : Chaque personnage doit offrir une expérience de jeu distincte.
- ❖ Lisibilité : Les animations (Idle, Walk, Attack, Hurt) ont été travaillées pour être distinctes afin de permettre aux joueurs de réagir rapidement aux actions adverses.
- ❖ Équilibrage : Un système de compromis (Pierre-Papier-Ciseaux) entre la portée, la vitesse de déplacement et la puissance d'attaque.

Le Chevalier (Équilibre / Tank)

C'est l'archétype du "All-rounder". Il sert de référence pour le jeu.

- Points forts : Très bonne portée avec son épée et une excellente santé (Tank).
- Points faibles : Vitesse de déplacement modérée.
- Capacité spéciale : Un combo rapide à deux coups.



Le Samouraï (Agilité / Projectile)

Inspiré du Japon féodal, il combine la vitesse au corps-à-corps et le harcèlement à distance.

- Points forts : Dash très rapide et lancer de Shuriken pour le contrôle de zone.
- Points faibles : Une santé très fragile.
- Capacité spéciale : Projectile Shuriken rapide (projo_shuriken_1).



Le Cromagnon (Brut / Distance)

Un personnage lourd qui utilise la force brute et des lances de jet pour maintenir l'adversaire à distance.

- Points forts : Puissance d'arrêt massive et forte santé.
- Points faibles : Temps de récupération lent après ses attaques.
- Capacité spéciale : Lancer de lance avec une trajectoire parabolique (projo_lance).



Le Robot (Technique / Zoner)

L'archétype technologique qui mise tout sur le contrôle du terrain avec ses explosifs.

- Points forts : Excellents dégâts de zone et maîtrise des distances.
- Points faibles : Vulnérable au corps-à-corps pur.
- Capacité spéciale : Projectile explosif (projo_1).



2.3 Répartition des tâches

Louis :

- Gestion de la boucle principale du moteur et des ticks (main.py et EngineTick.py)
- Développement du système d'affichage dynamique (qui s'adapte à la fenêtre) et d'actualisation de l'écran dans EngineRender.py
- Optimisation et maintenance de l'architecture réseau Peer-to-Peer (P2P) dans NetworkManager.py
- Sécurisation de la sérialisation et de l'envoi des paquets UDP (JSON)
- Implémentation de la prédiction des mouvements pour compenser la latence réseau
- Co-crédation et amélioration visuelle de l'interface utilisateur et des menus avec Adam

Adam :

- Création et complexification des personnages jouables avec plusieurs attaques, dash, double saut et bouclier (Player.py)
- Implémentation des différentes box de collision (Collision box, Hurt box et Hit box)
- Gestion de la configuration des contrôles et lecture du fichier keybindings.json (KeyBindings.py)
- Gestion de l'état des projectiles et de leurs trajectoires (Projectile.py)
- Co-crédation et amélioration visuelle de l'interface utilisateur et des menus avec Louis
- Équilibrage global du gameplay (dégâts, vitesse d'attaque, frames d'invulnérabilité)

Ioan :

- Direction Artistique : Création complète et intégration graphique de l'intégralité du *roster* de personnages (Cro-Magnon, Robot, Samouraï, Chevalier)
- Animation : Découpage des *spritesheets* et conception des différentes frames d'animation (Idle, Walk, Jump, Hurt, Attack 1 & 2) garantissant la fluidité visuelle des combats

- Recherche, édition et implémentation de tous les bruitages (SFX).
- Conception, design et développement du site internet vitrine officiel du projet *Rift Fighters*, servant de portail de présentation pour l'équipe et le jeu.

2.4 L'organisation de ce rapport

Ce rapport a été structuré de manière à suivre le fil chronologique du développement de notre jeu. Afin de retracer au mieux notre parcours et les défis rencontrés, nous avons découpé notre présentation en quatre grandes phases de développement :

- **La conception et les fondations techniques :** Cette première phase pose le cadre du projet. Elle présente l'équipe, notre philosophie de design, et détaille l'architecture modulaire de notre moteur développé de zéro.
- **L'implémentation du cœur du jeu et du réseau :** Cette deuxième étape plonge au cœur de la technique. Nous y expliquons la création des mécaniques de combat (physique, collisions) ainsi que le développement de notre architecture réseau en Peer-to-Peer.
- **La direction artistique et l'intégration graphique :** La troisième phase aborde notre processus créatif. Sans graphiste dans l'équipe, nous détaillons comment nous avons surmonté ce défi pour animer nos personnages et créer un rendu visuel dynamique.
- **Les finitions et le bilan final :** Enfin, la dernière phase est consacrée à l'aboutissement du projet. Elle couvre l'intégration de l'intelligence artificielle au comportement humanisé, l'habillage sonore, et se termine par une analyse critique de nos réussites techniques et des compromis réalisés au cours de l'année.

2.5 Outils et logiciels utilisés

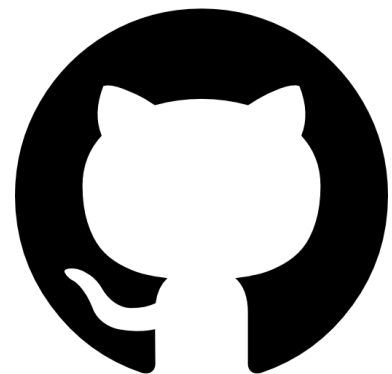
Tout au long de l'année, notre groupe de projet a utilisé divers outils et environnements pour développer notre jeu *Rift Fighters*, chacun ayant une utilité spécifique et essentielle à notre progression et à notre organisation.

- **Discord** a été la véritable tour de contrôle de notre communication.

Plutôt que de l'utiliser uniquement pour des appels vocaux, nous avons mis en place un serveur structuré et rigoureusement organisé. Nous y avons créé des salons textuels spécifiques et catégorisés pour trier nos idées de *game design*, stocker les informations techniques, et transférer nos ressources graphiques (sprites) de manière centralisée. Les discussions de groupe nous ont permis de résoudre rapidement les problèmes techniques en nous entraînant en temps réel.



- **GitHub** s'est révélé indispensable pour le stockage, la gestion de versions et l'organisation globale de notre code. Grâce à cet outil, nous avons pu suivre précisément les modifications apportées par chaque membre et collaborer de manière synchronisée. L'utilisation des branches (*branches*) a été vitale pour notre projet : elle nous a permis de programmer simultanément sur des fonctionnalités différentes (comme le réseau d'un côté et la physique de l'autre) sans jamais écraser le travail de nos camarades. GitHub a également assuré la sauvegarde sécurisée de tous nos fichiers, nous permettant de revenir à des versions précédentes en cas de bug critique.



- **Environnements de développement (IDE) :** Notre moteur de jeu étant intégralement développé en Python (via la bibliothèque PyGame), nous avons utilisé plusieurs éditeurs de code en python. Nous avons principalement travaillé sur **PyCharm et VSCode**.
- **Google Gemini (Création Graphique) :** N'ayant pas de graphiste de formation dans notre équipe, nous avons fait preuve d'ingéniosité pour la création de nos *assets* visuels. Notre processus de travail consistait à écrire nos idées sous forme de prompt à Gemini pour générer et affiner les *sprites* finaux en pixel-art, nous permettant d'obtenir des ressources graphiques cohérentes et de bonne qualité en un temps record.
- **Habillage Sonore :** Nous nous sommes appuyés sur la plateforme collaborative en ligne **Freesound**. Cette immense bibliothèque communautaire nous a permis de sourcer l'intégralité de nos effets sonores et bruitages (VFX) sous licence libre (Creative Commons). Qu'il s'agisse des lourds impacts lors des combats, des bruits de déplacement nerveux (*dash*), des retours sonores de l'interface (clics dans les menus) ou des attaques spécifiques à chaque combattant, tous les fichiers audio (au format .wav) gérés par notre moteur de son proviennent de cette base de données.

3. Conception et Architecture du Moteur From Scratch

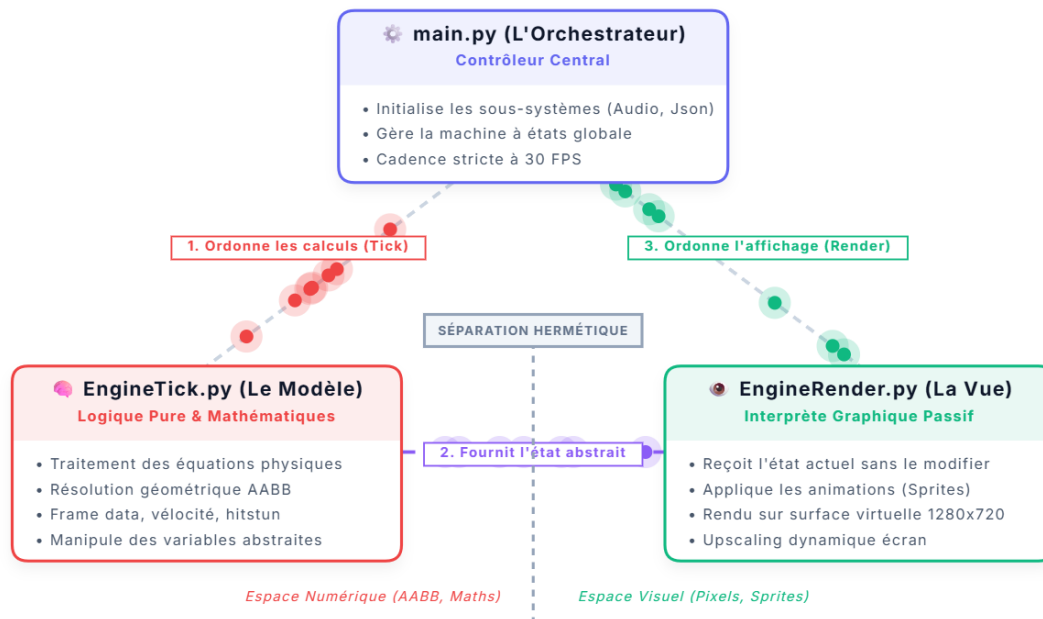
Pour assurer la viabilité, l'extensibilité et la maintenabilité à long terme de notre moteur de jeu propriétaire, l'un des choix d'ingénierie logicielle les plus cruciaux a été l'implémentation d'une architecture modulaire stricte. Cette dernière s'inspire directement d'une version simplifiée du patron de conception Modèle-Vue (Model-View Pattern). Dans un projet de cette envergure développé en partant de zéro (from scratch) sur PyGame, le principal risque technique réside dans l'apparition d'un code fortement couplé, où la logique algorithmique interne s'entremêle avec les primitives d'affichage graphique. Un tel couplage rendrait la traque des bugs chaotique et paralyserait l'intégration de fonctionnalités avancées comme la synchronisation réseau ou l'implémentation d'une intelligence artificielle.

Pour contourner cet écueil, notre structure logicielle sépare de manière hermétique la simulation physique et mathématique de sa représentation graphique à l'écran. Cette étanchéité est matérialisée par la répartition des responsabilités entre trois fichiers clés :

- `main.py` (L'Orchestrateur) : Il fait office de contrôleur central et de point d'entrée du programme. Sa responsabilité est double : d'une part, il gère l'initialisation des sous-systèmes (fenêtre, modules audio, liaisons de touches via le fichier de configuration locale `keybindings.json`) et la machine à états globale (transition entre l'écran d'accueil, le menu de sélection des personnages et la boucle de combat) ; d'autre part, il orchestre la boucle de jeu (Game Loop) principale à une cadence stricte et verrouillée de 30 images par seconde. À chaque itération de l'horloge système, il ordonne séquentiellement au modèle de mettre à jour les données, puis à la vue de rafraîchir l'affichage graphique.

- EngineTick.py (Le Modèle) : Ce module constitue le cerveau analytique et déterministe de Rift Fighters. Il est exclusivement responsable de la logique pure, du traitement des équations physiques et des calculs mathématiques appliqués aux entités. Il gère l'accumulation des forces (gravité), la modification des vecteurs de vitesse, le traitement des états des joueurs (déplacements, exécution de la frame data des attaques primaires et secondaires, durées de récupération, gestion du hitstun) ainsi que la résolution géométrique des collisions via l'algorithme AABB (Axis-Aligned Bounding Box).
- EngineRender.py (La Vue) : Ce script est dédié uniquement au sous-système graphique et à l'affichage visuel. Il ne contient aucune règle de jeu, aucune notion de points de vie et n'arbitre jamais si un coup doit infliger des dégâts. Son unique rôle est de lire l'état actuel des données physiques calculées par le modèle pour projeter les ressources graphiques (découpage des spritesheets, application des animations de course, de saut ou d'impact) sur une surface de rendu virtuelle fixe de 1280x720 pixels, avant d'effectuer l'upscaling final adapté au moniteur de l'utilisateur.

Isolation par Découplage Structurel



Le bénéfice fondamental de cette architecture réside dans l'indépendance totale du traitement des données vis-à-vis des coordonnées de pixels réelles de l'écran. Le module de logique (EngineTick.py) opère une simulation abstraite en arrière-plan : il manipule des variables numériques pures, déplace des rectangles virtuels et génère des projectiles sans jamais avoir connaissance de l'existence d'une fenêtre PyGame ou de la nécessité d'afficher un sprite. Par exemple, lorsqu'un joueur initie un saut, le modèle modifie uniquement sa position sur l'axe ordonné dans son repère de calcul bidimensionnel.

Réciproquement, le module de rendu (EngineRender.py) se comporte comme un interprète passif. Il reçoit en entrée la liste des entités vivantes et leurs boîtes englobantes actuelles, puis se contente de dessiner les éléments visuels correspondants sans avoir à connaître les conditions logiques sous-jacentes. Cette séparation stricte offre une flexibilité totale : si la direction artistique du jeu devait passer d'un style pixel-art 2D à un rendu entièrement différent, seul le fichier EngineRender.py nécessiterait une réécriture, tandis que l'intégralité du code de simulation physique, du

réseau et de l'équilibrage des personnages dans EngineTick.py resterait rigoureusement inchangée. Cette modularité a grandement facilité la répartition des tâches au sein de notre équipe et a permis d'optimiser le débogage en isolant immédiatement l'origine de chaque anomalie logicielle.

```

RiftFighters/
├─ main.py          # Point d'entrée du jeu : boucle principale à 30 FPS
├─ keybindings.json # sauvegarde des controles
├─ assets/          # Répertoire contenant toutes les ressources visuelles et
sonores
├─ Perso/           # Spritesheets et frames d'animations découpées par personnage
├─ Sounds/          # Bruitages (SFX) et musiques
├─ Stages/          # Images de fond des différentes arènes de combat
├─ src/             # Code source principal du jeu
├─ CoreEngine/      # Les briques fondamentales de notre moteur maison
├─   EngineTick.py  # les collisions et la mise à jour des entités
├─   EngineRender.py # dessine les surfaces PyGame, gère la caméra
├─   SoundManager.py # Chargement asynchrone et lecture des pistes audio
├─   Menus.py       # Interfaces interactives (sélection de personnages, d'arènes)
├─   GameUI.py      # Affichage du HUD en jeu (barres de vie, chronomètre)
├─   KeyBindings.py # Logique de remappage des touches
├─ Entities/        # Scripts définissant le comportement des objets en jeu
├─   Player.py      # Machine à états du joueur, gestion des inputs
├─   Projectile.py  # Classe mère et enfants pour les attaques à distance
├─   Platform.py    # Éléments de collision statiques (sol, murs)
├─   BotAI.py       # Intelligence Artificielle pour le mode solo
├─ Network/
├─   NetworkManager.py # Logique multijoueur : sockets UDP, UPnP
├─ Utils/
├─   UtilsFunctions.py # Fonctions mathématiques utilitaires

```

4. Implémentation du Cœur de Jeu : Physique et Réseau P2P

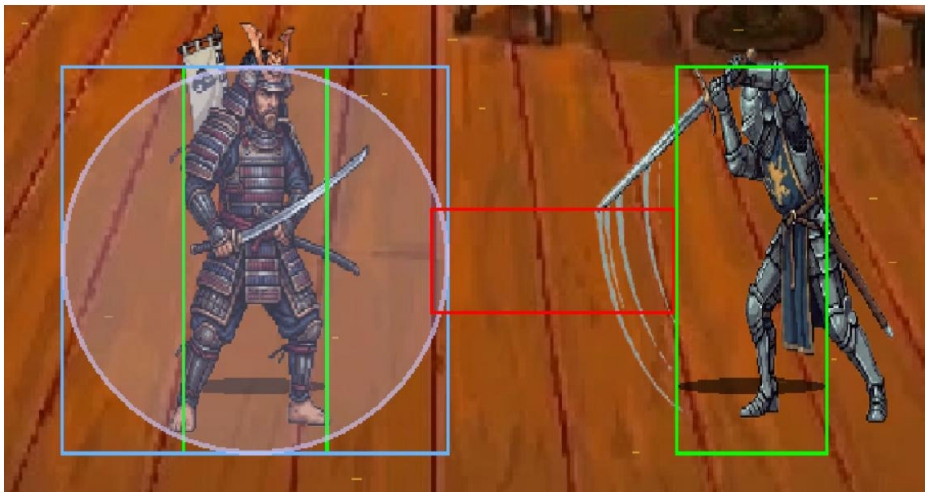
4.1 Les mécaniques de combat et la physique

- **Comment il me touche ? Hitboxes et Hurtboxes (Boîtes de Collision)**

Dans n'importe quel jeu de combat en 2D (de *Street Fighter* à *Super Smash Bros*), ce que le joueur voit à l'écran (le sprite ou le modèle 3D du personnage) n'est qu'une illusion visuelle. Le moteur physique résonne exclusivement sous forme de rectangles (ou de cercles) invisibles qui se superposent. On distingue fondamentalement deux types de boîtes. D'une part, la **Hurtbox** (boîte de vulnérabilité) : elle enveloppe le corps du personnage. Si une attaque touche cette zone, le personnage subit des dégâts. En général, elle est légèrement plus petite que le sprite pour éviter les frustrations où le joueur a l'impression d'avoir été touché "dans le vide" à cause d'un pixel de vêtement qui dépasse. D'autre part, la **Hitbox** (boîte d'attaque) : elle n'apparaît que pendant une fraction de seconde, attachée au poing, à l'épée ou au projectile du personnage lorsqu'il attaque. Quand une Hitbox entre en intersection avec la Hurtbox de l'adversaire, le moteur enregistre un "Hit". De plus, le moteur doit gérer la "collision de corps" (Collisionbox) pour empêcher deux personnages de se traverser simplement en marchant l'un vers l'autre.

Dans notre code, cette logique est implémentée via les @property dans `Player.py`. Au lieu de créer des rectangles statiques, on génère les boîtes dynamiquement à chaque *frame* (image). Pour la **Hurtbox** (qu'on appelle simplement `self.hitbox`), on utilise des ratios (`hitbox_width_ratio` fixé à 0.45 et `hitbox_height_ratio` à 0.85). Cela signifie que le corps physique du personnage est volontairement aminci de 55% en largeur et raboté de 15% en hauteur par rapport à son image réelle. Pour les **Hitboxes d'attaque** (`attack_hitbox` et `attack2_hitbox`), le rectangle est projeté devant le joueur en fonction de sa direction (`self.facing_right`). Ses dimensions et sa portée

(attack_reach, attack_height) dépendent des constantes définies dans ATTACK_DATA, ce qui permet au Samourai d'avoir une portée fine et longue, tandis que le Cromagnon frappe plus court mais plus haut. Enfin, la détection des collisions physiques est centralisée dans EngineTick.py. La méthode handle_entity_collisions calcule l'intersection entre les deux joueurs (collidirect). S'ils se chevauchent, le code calcule l'axe de superposition (overlap_left vs overlap_right) et repousse les joueurs de la moitié de cette distance chacun (correction = overlap_left // 2). Ainsi les joueurs ne peuvent pas se chevaucher.



- **Infliger et subir : Système de Vie, de Dégâts, Stun et Punition**

Retirer des points de vie (HP) n'est que la surface d'un système de dégâts. Quand une attaque touche, le jeu "gèle" momentanément les deux personnages. C'est ce qu'on appelle le **Hit Stop** (ou Attack Lag), qui donne cette sensation de "poids" au coup. Immédiatement après, l'attaquant reprend le contrôle, tandis que le défenseur entre en **Hit Stun** : une période de paralysie où il encaisse le coup. Si le Hit Stun est plus long que le temps que met l'attaquant à lancer une nouvelle attaque, cela crée un "Combo". Enfin, la notion de **Punish** (Punition) est cruciale : toute attaque possède trois phases : *Startup* (l'élan), *Active* (le moment où ça frappe) et *Recovery* (la récupération, où le personnage ramène son arme). Si un joueur esquive une attaque et frappe son adversaire pendant sa phase de *Recovery*, on dit qu'il le "punit", ce qui est souvent récompensé par des dégâts accrus.

Dans notre jeu chaque joueur possède `self.max_health = 100`. Lorsqu'une collision d'attaque est détectée par `EngineTick.py`, la fonction `take_damage(amount)` de la victime est appelée, et la méthode `apply_attack_lag()` de l'attaquant est déclenchée. Dans `Player.py`, l'attaquant subit un `ATTACK_LAG_FRAMES` de 8 frames (il est littéralement gelé, `self.is_moving = False`, son animation se fige). Le défenseur, s'il subit un coup normal, encaisse un `HIT_STUN_FRAMES` de 12 frames. De plus on a implémenté une mécanique de **Punition**. Le code vérifie si la victime était vulnérable via la propriété `was_punish = self.is_in_recovery` (c'est-à-dire si `attack_phase == "recovery"`). Si c'est le cas, les dégâts subis sont multipliés par deux (`PUNISH_DAMAGE_MULTIPLIER = 2.0`), le Hit Stun monte à 18 frames, et la phase d'attaque du défenseur est violemment annulée. La punition remonte jusqu'à `GameUI.py` via `_emit_punish_event`, déclenchant une bannière ("PUNISH !") pour récompenser la lecture de jeu du joueur.

- **Je vais tout bloquer : Le Bouclier et la mécanique de "Perfect Parry"**

La garde (ou bouclier) est l'outil défensif par excellence, mais il ne doit jamais être infailible pour éviter qu'un joueur ne se recroqueville indéfiniment (stratégie de la "tortue"). Les jeux gèrent cela avec des dégâts de gratte (*chip damage*), une jauge de rupture de garde, ou un délai de vulnérabilité lorsqu'on lâche le bouclier. De plus, depuis *Street Fighter III* et *Super Smash Bros*, la mécanique de garde parfaite (*Perfect Parry* ou *Perfect Shield*) récompense les joueurs qui activent leur bouclier à la milliseconde exacte de l'impact : cela annule totalement les dégâts et le recul, permettant une contre-attaque immédiate.



On a opté pour un bouclier actif (une bulle semblable à celle de *Smash Bros*, générée par la propriété `shield_hitbox`). Si le joueur maintient la touche bouclier au sol, il est immobilisé. Si une attaque frappe cette bulle, la fonction `take_damage` applique une réduction de dégâts (le joueur ne subit que 20% des dégâts initiaux, via `SHIELD_DAMAGE_RATIO`), et subit un léger stun de 6 frames. Cependant, nous avons intégré un délai de vulnérabilité : lorsqu'un joueur lâche son bouclier, il subit un `SHIELD_COOLDOWN_FRAMES` de 30 frames (soit une seconde entière à 30 FPS) pendant lesquelles il ne peut rien faire. C'est extrêmement punitif et empêche le spam défensif. La **Garde Parfaite (Perfect Shield)** est gérée

via un compteur d'âge du bouclier (`self.shield_age`). Si l'impact a lieu dans les 6 premières frames (`PERFECT_SHIELD_WINDOW`) suivant l'activation du bouclier, les règles changent drastiquement : les dégâts tombent à 0, le Hit Stun tombe à 0, le joueur est marqué `perfect_shielded = True`, et surtout, lorsqu'il relâchera le bouclier, le délai de 30 frames de cooldown sera annulé. Une bannière ("PERFECT") apparaîtra.

- **Les déplacements : Le Dash et le Jump**

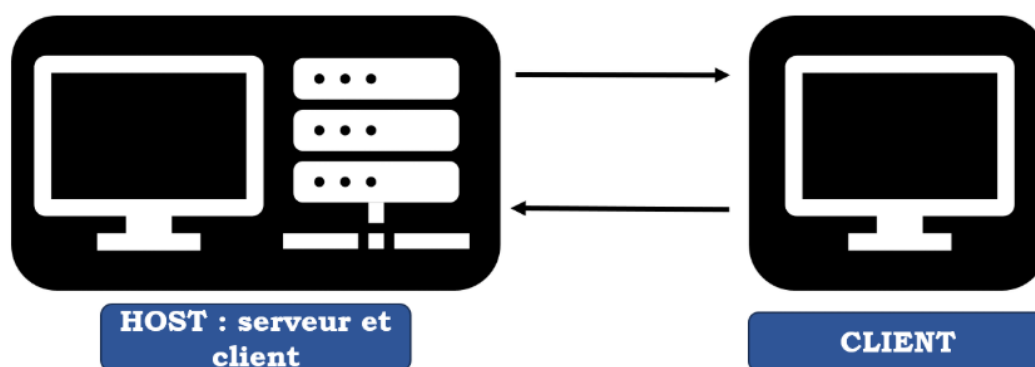
Le déplacement de base au sol est prévisible et lent. Pour dynamiser le rythme, les jeux de combat introduisent des options de mobilité. Au sol, cela se traduit par le *Dash* (la ruée), un mouvement rapide initié par un double-clic (avant-avant ou arrière-arrière) qui parcourt une distance fixe, généralement régulé par un délai de récupération (cooldown) pour empêcher le spam. Dans les airs, la maîtrise de l'espace vertical est tout aussi vitale. Le saut permet d'attaquer sous de nouveaux angles, et l'ajout d'un *double saut* offre la possibilité cruciale de modifier sa trajectoire en plein vol, que ce soit pour feinter un adversaire (bait), allonger une distance de fuite, ou contourner un projectile.

Dans notre jeu, la mobilité horizontale et verticale est développée à l'aide d'un dash et d'un double saut. Pour le système de Dash, on compte le nombre d'appuis sur les touches de direction avec un chronomètre (`left_tap_timer`). Si le deuxième appui survient en moins de 15 frames, le Dash est validé. Le moteur remplace alors la vitesse standard par un déplacement absolu très rapide (200 pixels en 8 frames). À la fin du mouvement, on impose un `DASH_COOLDOWN` de 20 frames, judicieusement traduit par une petite barre orange d'endurance sous le joueur pour rendre la limitation visuellement intuitive.

Pour les airs, chaque joueur possède un compteur `self.jumps_remaining` initialisé à 2. La grande force de cette implémentation réside dans la détection du « front montant » de la touche (l'expression `jump_pressed and not self.jump_prev`). Cela garantit qu'un joueur qui laisse la touche saut enfoncée ne va pas consommer ses deux sauts accidentellement en un quart de seconde. Le premier saut applique une impulsion complète (`self.jump_strength`), tandis que le double saut est bridé à 85% de cette force (`int(self.jump_strength * 0.85)`). Cela empêche les personnages de sortir du cadre supérieur de l'écran trop facilement. Enfin, dès que `EngineTick.py` détecte une collision entre la *hitbox* du joueur et la plateforme du sol, on remet automatiquement les compteurs à zéro et les deux sauts sont rechargés pour la prochaine manœuvre.

4.2 L'architecture réseau

Le multijoueur de *Rift Fighters* repose sur une architecture en Peer-to-Peer (P2P) autoritaire plutôt que sur un modèle client-serveur, un choix technologique directement motivé par des raisons de budget afin d'éviter les coûts de location de serveurs dédiés. Dans ce système, l'un des joueurs héberge la session (Hôte) et lie son socket réseau au port local 6767 en se mettant sur écoute. Le second joueur (Client) renseigne manuellement l'adresse IP de l'Hôte dans les menus (qu'il s'agisse d'une IP locale en 192.168.x.x pour du LAN, ou publique pour du WAN) et envoie un premier datagramme de "ping" pour établir un tunnel de communication bilatéral.



Une fois la connexion établie, le trafic transite exclusivement via le protocole UDP. Dans le cadre d'un jeu de combat, ce choix est impératif : contrairement au TCP, l'UDP n'exige pas d'accusé de réception et ne bloque pas l'exécution du programme en cas de perte de paquet. Le flux continu de données permet ainsi de maintenir rigoureusement la fréquence de 30 FPS constants.

Afin de simplifier drastiquement l'expérience utilisateur et de rendre le jeu en ligne "Plug and Play", notre module NetworkManager automatise le passage des sécurités réseau. Dans un premier temps, il utilise la bibliothèque `miniupnpc` pour envoyer une requête IGD (Internet Gateway Device) au routeur du joueur Hôte. Ce protocole UPnP permet de configurer dynamiquement la box internet

pour rediriger le port 6767 vers la machine locale, contournant ainsi le NAT. Dans un second temps, pour éviter le blocage natif du pare-feu Windows Defender, le jeu sollicite dynamiquement les droits administrateurs via l'API Windows (`ctypes.windll.shell32.ShellExecuteW`). Une fois l'invite UAC validée par le joueur, le programme exécute silencieusement une commande `netsh advfirewall` afin de créer sa propre règle d'exception pour le trafic entrant.

Enfin, pour offrir des combats fluides et éliminer le fameux effet d'élastique (*rubber-banding*) causé par la latence, nous avons conçu un système avancé de "Prédiction Client" et de "Réconciliation". Le joueur local applique et affiche la physique de ses propres mouvements instantanément, garantissant un "zéro input lag". Parallèlement, le moteur garde ces actions en mémoire. Lorsque l'Hôte renvoie l'état "officiel" de la partie, le Client compare les données : si un décalage est détecté, le jeu corrige la position du personnage de manière invisible en une fraction de milliseconde et recalcule les frames manquantes, assurant une synchronisation parfaite à l'écran.

Modèle P2P Autoritaire avec Prédiction Client

Garantie d'un zéro input lag sans serveurs centraux coûteux.



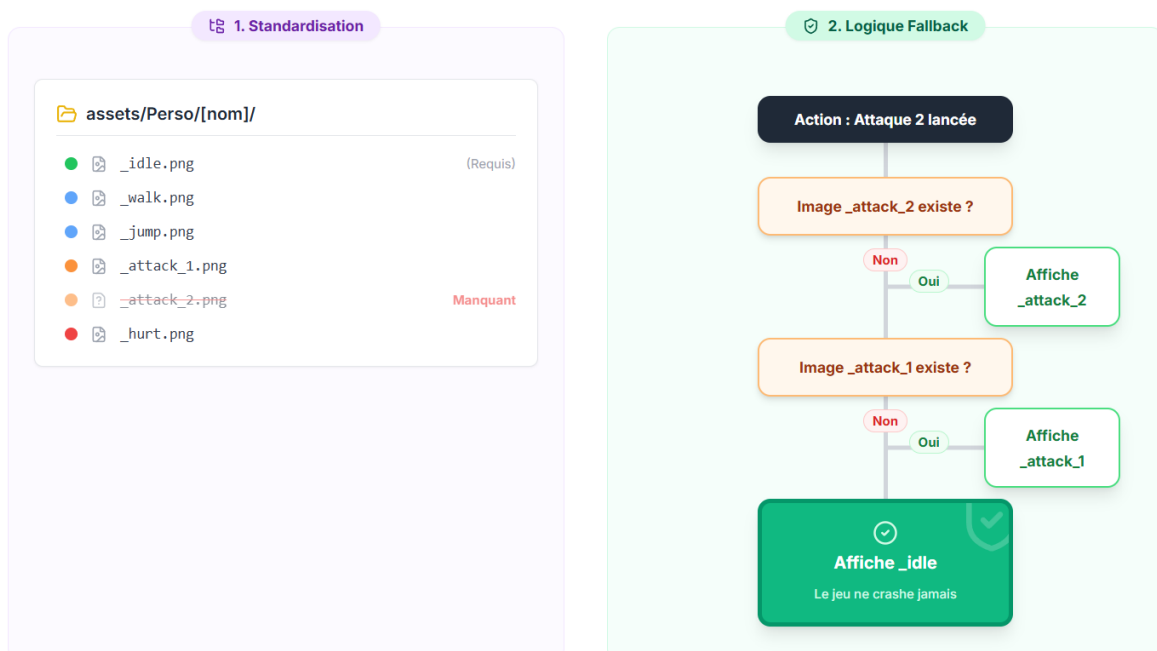
5. Direction Artistique et Intégration Visuelle

L'un des défis majeurs de *Rift Fighters* fut de proposer une expérience visuelle dynamique et lisible, malgré l'absence de graphistes dans notre équipe et un nombre très limité de ressources (assets). Nous avons dû concevoir un système de rendu intelligent pour donner vie à nos personnages et à nos arènes.

5.1. Standardisation des Assets et Identité Visuelle

Pour pallier notre manque d'expérience en design, nous avons généré nos personnages (Chevalier, Cromagnon, Robot, Samouraï) en *pixel-art* à l'aide de l'intelligence artificielle générative. Pour que notre moteur puisse traiter ces images de manière générique, nous avons mis en place une convention de nommage stricte. Chaque personnage possède un set minimal de 6 images statiques : `_idle` (repos), `_walk` (marche), `_jump` (saut), `_attack_1`, `_attack_2` et `_hurt` (dégâts).

Le moteur charge automatiquement ces images en remplaçant simplement le suffixe du nom du personnage. Si une image manque (par exemple `_attack_2`), le code effectue un "fallback" silencieux vers une autre image existante (`_attack_1`), garantissant que le jeu ne crashera jamais à cause d'un asset manquant.



5.2. L'Illusion du Mouvement : Animer avec très peu d'images

La véritable force de notre rendu réside dans la manière dont nous simulons des animations fluides alors que nous ne possédons que des images statiques.

Le système de priorité d'affichage : À chaque frame (tick), le moteur évalue l'état du joueur et choisit l'image appropriée selon un ordre de priorité strict :

1. **L'étourdissement (`_hurt`) :** Priorité absolue si le joueur vient de recevoir un coup.
2. **Les attaques (`_attack_2` puis `_attack_1`) :** Affichent la pose de frappe tant que la hitbox est active.
3. **Le saut (`_jump`) :** Affiche la pose aérienne si le joueur n'est pas en contact avec le sol.
4. **Le mouvement (`_walk` / `_idle`) :** Si le joueur marche ou effectue un *dash*.

L'astuce de la marche (Walk Cycle) : Plutôt que d'avoir une feuille de sprites (spritesheet) complexe avec 10 frames d'animation pour la marche, nous créons l'illusion du mouvement en alternant très rapidement seulement deux images : l'image de repos (`_idle`) et l'image de marche (`_walk`). Nous utilisons un chronomètre interne (`anim_timer`) qui s'incrémente à chaque itération de la boucle de jeu. Grâce à une division modulo, le code bascule entre l'image `_walk` et l'image `_idle` toutes les 4 frames. À 30 images par seconde, cette simple alternance donne l'impression que le personnage piétine et avance de manière dynamique.

Le retournement dynamique (Flipping) : Pour économiser de la mémoire et diviser par deux le nombre d'images nécessaires, nous ne dessinons que les personnages tournés vers la droite. Si le joueur se déplace vers la gauche (ou si l'adversaire passe derrière lui), nous utilisons la fonction `pygame.transform.flip(image, True, False)` pour inverser l'image horizontalement en temps réel juste avant de l'afficher.

5.3. Effets Visuels (VFX) et "Game Feel"

Pour compenser la rigidité des images statiques, nous avons massivement investi dans les effets visuels générés par le code (et non par des images), ce qui renforce considérablement l'impact des combats :

- **Feedback des dégâts (Particules) :** Lorsqu'un joueur perd des points de vie, le moteur de rendu intercepte cette baisse et déclenche la fonction `spawn_hit_particles`. Cette fonction génère une vingtaine de petits cercles de couleurs vives qui explosent dans toutes les directions (calculées via des formules de trigonométrie $\cos$ et $\sin$) et retombent sous l'effet de la gravité. Cela rend les coups extrêmement satisfaisants.
- **Effets de déplacement (Dash) :** Lors d'un *dash*, au lieu de créer une nouvelle animation, nous copions l'image du joueur et nous lui appliquons une transparence ($\alpha = 160$). Cela crée un effet de fantôme et de vitesse très lisible pour l'adversaire.
- **Le Bouclier :** Plutôt que de dessiner un bouclier statique, le code génère un cercle translucide dynamique au-dessus du joueur, accompagné de barres de rechargement (cooldowns) directement dessinées sous les pieds des personnages avec les primitives géométriques de Pygame.
- **Ombres dynamiques :** Nous dessinons une ellipse noire translucide sous chaque personnage. La taille de cette ombre rétrécit proportionnellement à la hauteur du saut du personnage (`dist_to_floor`), ce qui aide les joueurs à évaluer la profondeur et le point de chute lors des combats aériens.

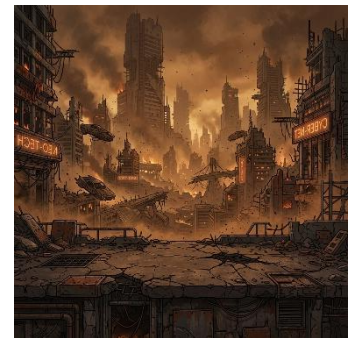


5.4. L'Ambiance des Arènes (Stages)

Enfin, pour donner vie à nos arrière-plans statiques (Cave, Futur, FarWest, etc.), nous avons programmé des systèmes de particules uniques pour chaque carte dans l'EngineRender :

- **Stage "Futur / Lab"** : Génération d'une pluie de lasers néons qui tombent verticalement de l'écran.
- **Stage "Cave"** : Des particules lumineuses (spores ou lucioles) flottent de bas en haut avec un mouvement d'oscillation géré par une fonction sinusoïdale (math.sin), avec un rayon lumineux clignotant.
- **Stage "FarWest"** : Un filtre orange translucide recouvre l'écran, combiné à des traits horizontaux très rapides pour simuler une tempête de sable.

Toutes ces techniques mathématiques et algorithmiques nous ont permis de créer un rendu visuel riche et "juicy" tout en contournant notre principal obstacle : le manque de compétences en dessin.



6. Finitions Techniques : Intelligence Artificielle et Système Sonore

Dans la dernière ligne droite de notre développement, nous avons concentré nos efforts sur les éléments manquants de notre prototype. Tout d'abord, nous avons implémenté une Intelligence Artificielle (Bot) permettant aux joueurs de s'entraîner en solo face à un adversaire capable d'effectuer des mouvements et des attaques cohérents. Ensuite, nous avons intégré un habillage sonore complet, qui avait pris beaucoup de retard lors des étapes précédentes. Enfin, nous avons amélioré l'expérience utilisateur en peaufinant l'aspect visuel des menus et en ajoutant un onglet détaillant les règles du jeu.

6.1 Une Intelligence Artificielle Humanisée (Le Bot)

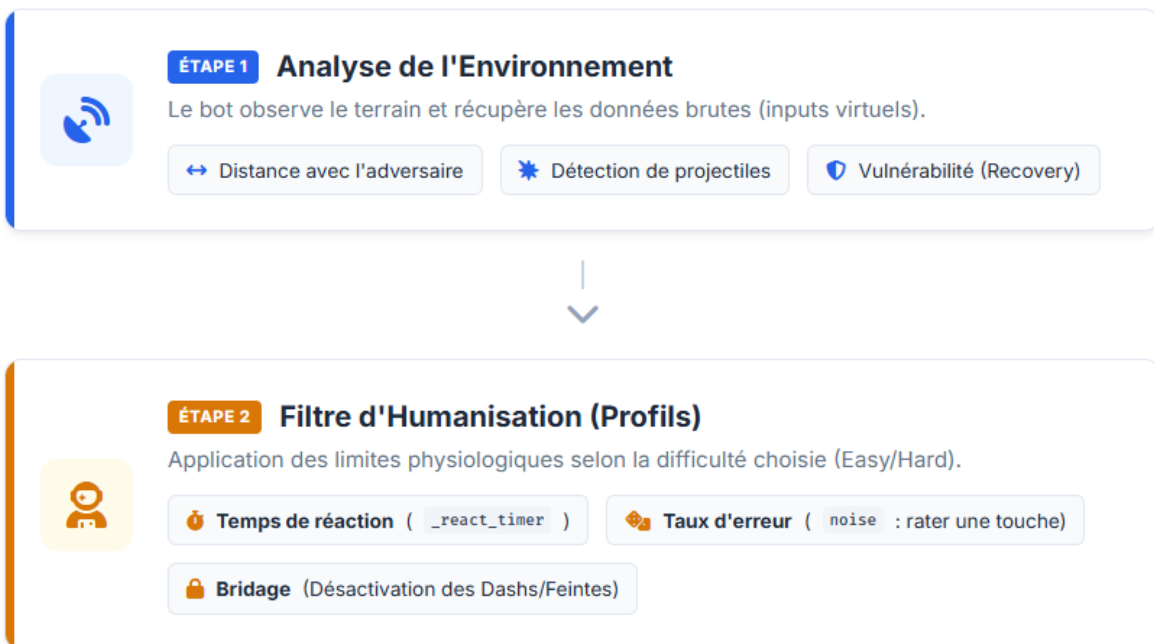
Pour offrir un mode solo intéressant, nous avons codé notre IA en nous basant sur un modèle classique dans le domaine du jeu vidéo : la Machine à États Finis (FSM - Finite State Machine). Notre parti pris en matière de *Game Design* a été de refuser la facilité : au lieu de concevoir une IA qui "triche" en lisant directement les actions du joueur pour y répondre instantanément, notre algorithme simule le comportement humain en appuyant sur une manette virtuelle, avec ses propres temps de réaction et ses propres erreurs.

1. Humanisation de l'IA

La première grande force de ce bot est sa capacité à paraître humain, gérée par le dictionnaire PROFILES. Au lieu de simplement rendre le bot "EASY" plus faible en termes de dégâts, on a limité ses capacités de ce dernier :

- **Le temps de réaction (reaction) :** Lorsqu'un événement se produit (comme l'adversaire qui rate une attaque), le bot décide de changer d'état. Cependant, il ne le fait pas immédiatement. Le `_react_timer` le force à attendre un certain nombre de *frames*. À "EASY", il met 28 images (presque une seconde) à réagir. À "HARD", il réagit en 3 images, ce qui le rend redoutable.

- **Le taux d'erreur (noise) :** À chaque *frame*, le bot tire un nombre aléatoire. Si ce nombre tombe sous le seuil de noise (32% en EASY, seulement 1% en HARD), le bot "rate" sa boucle de réflexion et maintient simplement les touches de l'image précédente. Cela crée des déplacements organiques et de légères maladresses typiques d'un joueur humain débutant.
- **Le bridage mécanique :** Les profils désactivent certaines mécaniques pour les niveaux inférieurs (`use_dash = False`, `use_shield = False`). Un bot "EASY" va simplement marcher vers son adversaire et utiliser l'attaque de base, tandis qu'un bot "NORMAL" ou "HARD" aura le droit d'utiliser des feintes (`feint`), des esquives (`use_dash`), et de punir spécifiquement les vulnérabilités de l'adversaire (`punish`).



2. L'Analyse de l'Environnement (La Boucle tick)

Soixante fois par seconde, le bot observe le terrain dans la méthode `tick(self, opponent)`. Il récolte des données brutes, exactement comme ce qu'un joueur voit à l'écran :

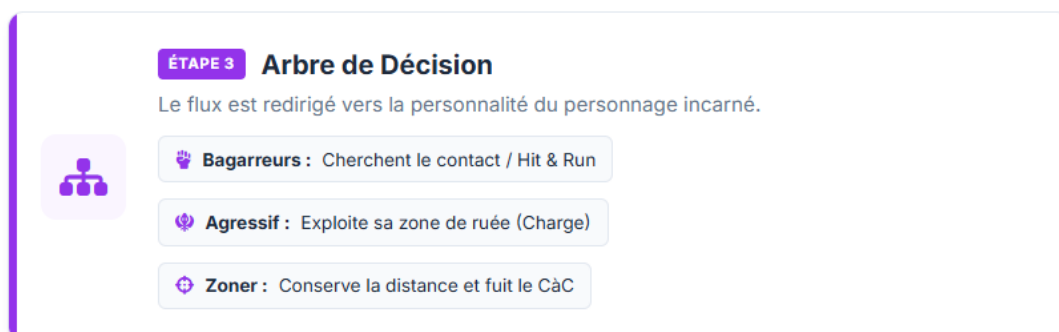
- Quelle est la distance qui le sépare de l'adversaire (dist) ?
- L'adversaire est-il en train de tirer un projectile (opp_proj, détecté via la méthode _opp_shooting) ?
- L'adversaire est-il vulnérable après une attaque ratée (opp_recovery) ?

Une fois ces informations récoltées, et si son temps de réaction est écoulé, l'IA redirige le flux vers un arbre de décision spécifique au personnage joué.

3. Les Personnalités de Combat (Arbres de Décision)

Chaque combattant possède sa propre méthode de décision (`_decide_cromagnon`, `_decide_robot`, etc.), ce qui leur donne une "personnalité" unique sur le terrain :

- **Le Cromagnon et le Samourai (Les Bagarreurs)** : Ils cherchent activement le corps-à-corps. Si l'adversaire tire, ils lèvent le bouclier. S'il est vulnérable, ils attaquent. S'ils sont à la bonne distance, ils frappent, mais le Samourai a une probabilité plus élevée de fuir (`S_RETREAT`) si l'adversaire est trop proche, simulant une stratégie de *Hit & Run* (Frapper et fuir).
- **Le Chevalier (L'Agressif)** : Sa logique exploite pleinement son attaque secondaire (la ruée). Si l'adversaire se trouve entre la portée de mêlée et la portée de sa charge (`MELEE_ATK_RANGE <= dist <= RUSH_RANGE`), le bot déclenche l'attaque 2 pour fondre sur sa cible et la surprendre.
- **Le Robot (Le Zoner)** : Sa logique est totalement inversée. Il ne cherche pas à s'approcher (`S_APPROACH`) sauf s'il est extrêmement loin. Si l'adversaire entre dans sa zone de confort (`ROBOT_MIN` à `ROBOT_MAX`), il tire en boucle. Si l'adversaire s'approche trop près (en dessous de `ROBOT_MIN`), l'état du bot passe en `S_REPOSITION` pour fuir et reprendre de la distance.



4. La Machine à États (L'Exécution)

Une fois la décision prise, le bot entre dans un état précis (défini par les constantes `S_APPROACH`, `S_ATTACK`, `S_SHIELD`, etc.). La méthode `_run_state` traduit cet état en action en jeu :

- **Génération des inputs** : La fonction `_n()` crée un dictionnaire propre où toutes les touches (gauche, droite, saut, attaque, etc.) sont sur `False`. Le bot va ensuite passer certaines touches sur `True` selon son état.
- **Gestion des mouvements complexes** : Si le bot décide de dasher (esquiver), la méthode `_start_dash` génère une séquence de touches artificielle `[True, False, True, False]`. C'est une simulation parfaite d'un humain qui double-clique rapidement sur une flèche directionnelle. Cette séquence prend la priorité absolue lors des frames suivantes.
- **Gestion des attaques** : La méthode `_do_attack` choisit s'il faut presser `attack` ou `attack2` en fonction de la distance calculée, puis simule un "cooldown" mental (`_atk_cd`) basé sur les frames d'animation réelles du personnage (`startup + active + recovery + une marge de sécurité`) pour éviter que le bot ne tente de spammer le bouton pendant que son personnage fait déjà une action.

À la toute fin du cycle, le bot retourne ce dictionnaire d'inputs à l'instance de `Player.py` (`p2.update_inputs(bot_inputs)` dans le `main`). Le jeu ne fait absolument aucune différence entre ces touches générées par l'algorithme et celles pressées par un véritable joueur sur son clavier.

ÉTAPE 4 Exécution (Machine à États)

Traduction de la décision en signaux physiques pour le moteur.



 Génération d'un dictionnaire `_n()`

 Simulation double-clic (ex: `_start_dash`)

 Cooldowns mentaux calqués sur les animations

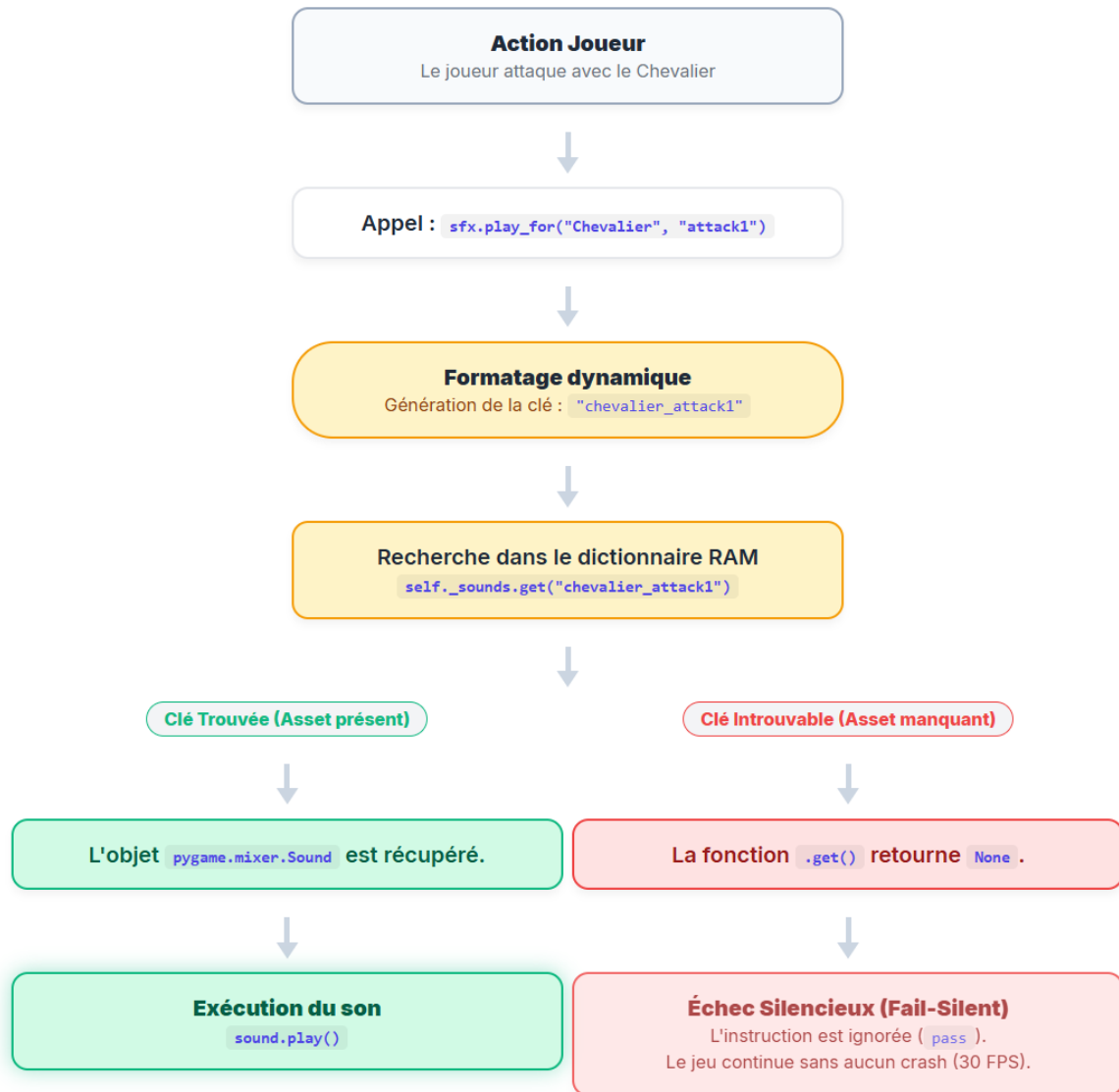
6.2 Le Système Sonore Centralisé

L'implémentation des sons dans le jeu repose sur un gestionnaire centralisé et sécurisé, structuré autour du modèle de conception Singleton (via l'écrasement de la méthode `__new__`). Cela garantit qu'une seule instance du `SoundManager` contrôle l'intégralité du flux audio, évitant ainsi les conflits d'accès et la surcharge de la mémoire.

Cette approche se divise en 3 points:

- **Préchargement et faible latence** : Dès sa création, le gestionnaire initialise le module `pygame.mixer` avec un petit buffer (512) pour s'assurer que les sons réagissent instantanément aux actions nerveuses d'un jeu de combat. Tous les fichiers `.wav` trouvés dans `assets/Sounds/` sont immédiatement chargés et indexés dans un dictionnaire.
- **Séparation des contextes** : La logique distingue les sons globaux (clic, dash, bouclier) appelés via un simple `play("nom")`, des sons spécifiques aux combattants. La méthode dynamique `play_for("Personnage", "action")` concatène les chaînes pour trouver la bonne piste audio (ex: `cromagnon_attack2.wav`).

- Tolérance aux pannes: L'architecture est totalement permissive face aux assets manquants. Si un fichier audio est manquant ou ne réussit pas à charger, le code gère l'erreur silencieusement. Le jeu continuera de fonctionner parfaitement, simplement sans émettre de bruit, sans jamais provoquer de crash.



7. Bilan du projet

L'heure est au bilan. Après plusieurs mois de développement intenses, marqués par des imprévus mais aussi de belles victoires techniques, il est essentiel de confronter notre cahier des charges initial au résultat final de *Rift Fighters*. Voici l'état des lieux de nos objectifs :

Personnages et Entités : Objectif atteint (100%) L'intégralité de nos ambitions concernant les combattants a été concrétisée. Nos quatre personnages aux styles de jeu asymétriques (Cromagnon, Samouraï, Robot et Chevalier) sont parfaitement jouables. Ils possèdent tous leurs propres statistiques, leurs attaques uniques, leurs mécaniques spécifiques (projectiles, boucliers) et l'intégralité de leurs animations a été intégrée avec succès.

Interface (GUI) : Objectif atteint (100%) L'interface utilisateur est complète, lisible et totalement fonctionnelle. De la navigation dans les menus principaux et la sélection des personnages, jusqu'au HUD (Head-Up Display) en plein combat (barres de vie, timer, indicateurs de cooldown du *dash* et du bouclier), tout répond parfaitement aux normes d'un véritable jeu de combat.

Moteur et Maps : De grandes réussites et quelques compromis

- **Ce qui a été réussi :** Nous sommes particulièrement fiers d'avoir construit un moteur de jeu *from scratch*. La gestion de la physique (gravité, sauts), l'exactitude des collisions (*hitboxes*) et notre système de rendu visuel (effets de particules, ombres dynamiques) fonctionnent de manière très fluide.

- **Ce qui n'a pas pu être fait :**

- **Un Mode Histoire et un Tutoriel Guidé :** Bien que nous ayons réussi à implémenter une intelligence artificielle pour permettre l'entraînement en solo, la création d'une campagne narrative et d'un tutoriel interactif pas-à-pas a été écartée. Ces modes exigeaient un système de *scripting* d'événements et un volume de contenu trop importants pour le temps qui nous était imparti.
- **Un véritable système de combos :** Notre moteur physique gère très bien l'étourdissement (*hit stun*) et les fenêtres de punition, mais nous n'avons pas eu le temps d'implémenter un système d'enchaînement de coups complexe (avec un affichage du compteur de combos et un calcul dynamique de réduction des dégâts).
- **Un profil et des statistiques utilisateur :** Nous avions pour ambition de créer un système de sauvegarde locale pour enregistrer l'historique et les statistiques du joueur (ratio de victoires/défaites, personnage le plus joué, temps de jeu total). Le développement de cette persistance des données a finalement été mis de côté pour nous concentrer sur la stabilité du multijoueur.
- **Un mélangeur de sons (Audio Mixer) :** Si notre gestionnaire de sons centralisé remplit parfaitement son rôle technique, il nous a manqué le temps nécessaire pour concevoir l'interface utilisateur (UI) d'un mélangeur audio. Par conséquent, il n'est actuellement pas possible pour le joueur d'ajuster le volume de la musique indépendamment de celui des bruitages (SFX) depuis les menus.

Réseau Multijoueur : Une prouesse technique perfectible

- **Ce qui marche très bien :** Nous avons fait le choix technique d'une architecture réseau en *Peer-to-Peer* (P2P) pour des raisons de budget. Ce modèle s'est avéré redoutablement efficace pour la synchronisation des parties : la prédiction des mouvements et l'enregistrement des frappes fonctionnent très bien, offrant un combat fluide entre deux machines distantes.
- **Ce qui reste problématique :** Le talon d'Achille de notre système réseau réside dans la gestion des erreurs et des fermetures inattendues de *sockets*. Actuellement, la déconnexion brutale d'un joueur en pleine partie n'est pas gérée de manière sécurisée (absence d'un *timeout* propre) et provoque le crash du jeu. C'est un aspect complexe que nous n'avons pas eu le temps de fiabiliser avant cette soutenance.

8. Conclusion

Bilan final :

Au terme de ce projet, nous pouvons affirmer avec fierté que Rift Fighters correspond fidèlement à notre vision initiale : créer un jeu de combat 2D nerveux où s'affrontent des figures historiques et futuristes issues de failles temporelles. Le cœur du jeu a été respecté, et nous sommes même allés plus loin que nos attentes initiales sur certains aspects techniques, notamment avec le développement complet de notre propre système de synchronisation multijoueur en Peer-to-Peer (P2P) ou l'implémentation d'une intelligence artificielle pour le mode solo.

Bilan humain et technique :

Cette année de développement a été extrêmement formatrice à tous les niveaux. Techniquement, nous avons dû sortir de notre zone de confort pour appréhender une architecture réseau complexe et maîtriser le travail collaboratif en surmontant de très nombreux conflits Git. Sur le plan humain, la perte de deux membres en cours de route a été un véritable test de résilience. Nous avons appris à nous réorganiser dans l'urgence, à communiquer plus efficacement et à gérer une charge de travail colossale, particulièrement lors de la fin d'année marquée par l'enchaînement de nos partiels et d'autres travaux d'envergure comme le projet minimake.

Le mot de la fin :

Ce projet SAE J3D nous a poussés à nous dépasser. Rift Fighters n'est pas seulement un exercice académique ; c'est un jeu complet, fonctionnel et abouti, qui témoigne de notre capacité d'adaptation et de notre persévérance face aux obstacles. Nous sommes très fiers du résultat accompli par notre équipe réduite.